

On-Chain DEX Trading Analytics — Build Process

From raw Dune Analytics query results to a PostgreSQL-backed Power BI dashboard. This document shows the data pipeline end-to-end: source query, extraction, database load, and the SQL views powering each dashboard page.

Step 1 — Source Query on Dune Analytics

Raw trader-level aggregates were computed directly against the public dex.trades table on Dune Analytics (query executed on the free tier, query ID 8571898). The query aggregates trades from the trailing 90-day window per unique trader address (tx_from), filtering out decoding anomalies (amount_usd outliers common in raw DEX data).

< Usage

Event details

2026-09-01 08:49

Query ID

8571898

Execution ID

01M1_FQXQ

Status

Success

Execution details

Record type	Credits	Product	User name	Duration
Query Execution	7.6966	Data Hub	hui_satoschi	21s
CPU seconds				
11m				

SQL text

```
SELECT
  tx_from AS trader,
  MIN(block_time) AS first_trade,
  MAX(block_time) AS last_trade,
  COUNT(*) AS total_trades,
  SUM(amount_usd) AS total_volume_usd
FROM dex.trades
WHERE block_time >= NOW() - INTERVAL '90' DAY
  AND amount_usd > 0
  AND amount_usd < 1000000 -- отсекаем аномальные/бракованные сделки
GROUP BY tx_from
HAVING COUNT(*) > 1
ORDER BY total_volume_usd DESC
LIMIT 500
```

Execution metadata

Cell count	Column count	Execution attempts	Execution duration seconds	Execution pending milliseconds
2500	5	2	21	24712
Planning time seconds	Result set bytes	Row count	Performance	
0.35926	37000	500	Medium	

Execution log from Dune: 500 rows, 5 columns, ~21 seconds runtime, confirming the query ran successfully against live on-chain data.

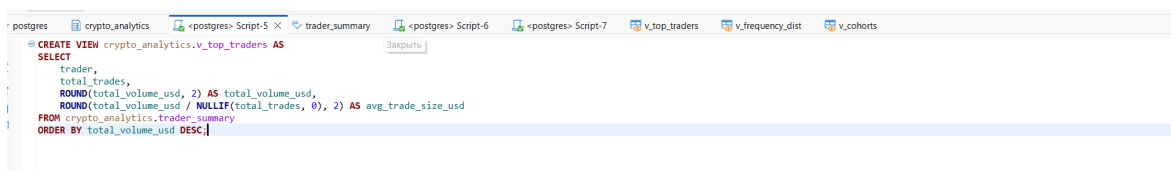
Step 2 — Extraction & Load into PostgreSQL

Query results were exported from Dune (via manual copy from the results grid, with locale/number-formatting corrected in Excel) and imported into a dedicated PostgreSQL database (**crypto_portfolio**, schema **crypto_analytics**), isolated from any other project database. This mirrors a typical production pattern: raw data lands in a staging table (**trader_summary**), and all downstream reporting logic lives in SQL views on top of it — not in Power BI itself — so the transformation logic is transparent, version-controllable, and reusable outside of the BI tool.

Step 3 — SQL Views Powering Each Dashboard Page

View 1 — v_top_traders (Top Traders page)

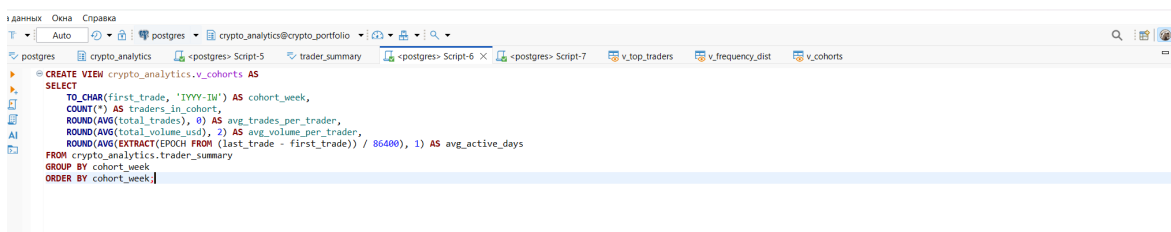
Ranks all traders by total volume and derives average ticket size — the metric used to separate high-frequency bots from manual, high-value traders on the scatter chart.



```
CREATE VIEW crypto_analytics.v_top_traders AS
SELECT
  trader,
  total_trades,
  ROUND(total_volume_usd, 2) AS total_volume_usd,
  ROUND(total_volume_usd / NULLIF(total_trades, 0), 2) AS avg_trade_size_usd
FROM crypto_analytics.trader_summary
ORDER BY total_volume_usd DESC;
```

View 2 — v_cohorts (Cohorts page)

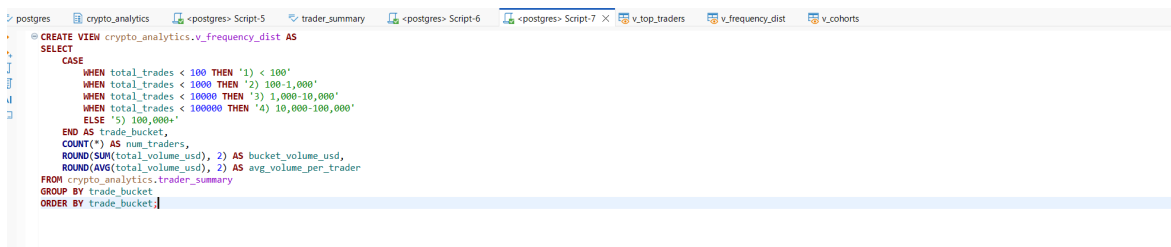
Groups traders by the ISO week of their first observed trade within the 90-day window, computing cohort size, average trading frequency, average volume, and average active lifespan (retention proxy) per cohort.



```
CREATE VIEW crypto_analytics.v_cohorts AS
SELECT
  TO_CHAR(first_trade, 'YYYY-IW') AS cohort_week,
  COUNT(*) AS traders_in_cohort,
  ROUND(AVG(total_trades), 0) AS avg_trades_per_trader,
  ROUND(AVG(total_volume_usd), 2) AS avg_volume_per_trader,
  ROUND(AVG(EXTRACT(EPOCH FROM (last_trade - first_trade)) / 86400), 1) AS avg_active_days
FROM crypto_analytics.trader_summary
GROUP BY cohort_week
ORDER BY cohort_week;
```

View 3 — v_frequency_dist (Frequency Distribution page)

Buckets traders into five activity tiers by trade count and aggregates trader count and volume per tier — revealing the concentration of volume among high-frequency addresses.



```
CREATE VIEW crypto_analytics.v_frequency_dist AS
SELECT
  CASE
    WHEN total_trades < 100 THEN '1' < 100'
    WHEN total_trades < 1000 THEN '2' 100-1,000'
    WHEN total_trades < 10000 THEN '3' 1,000-10,000'
    WHEN total_trades < 100000 THEN '4' 10,000-100,000'
    ELSE '5' 100,000+'
  END AS trade_bucket,
  COUNT(*) AS num_traders,
  ROUND(SUM(total_volume_usd), 2) AS bucket_volume_usd,
  ROUND(AVG(total_volume_usd), 2) AS avg_volume_per_trader
FROM crypto_analytics.trader_summary
GROUP BY trade_bucket
ORDER BY trade_bucket;
```

Pipeline summary: Dune Analytics (SQL on public on-chain data) → manual/API export → PostgreSQL (staging table + SQL views) → Power BI (direct PostgreSQL connection, three-page dashboard).